SDG BLOCKCHAIN ACCELERATOR

Mentor Feedback Report – xCapit (SHELTER)

Program: EMURGO – UNDP Blockchain Accelerator (Cohort 2)
Project: xCapit – SHELTER
Focus Area: Humanitarian Aid, Financial Inclusion, Offline / Low-Tech Wallets
Mentorship Focus: Technical Architecture, Security, Operational & Pilot Readiness
Total Mentoring Sessions: 6
Mentor: Cong Lee

1. Context at Start of Mentorship

xCapit joined Cohort 2 with a compelling and grounded problem statement: humanitarian aid distribution in environments where beneficiaries do not have smartphones, internet access, or bank accounts. The team had already invested significant effort in understanding field realities in Malawi and similar contexts.

At the beginning of the mentorship:

- The product logic was clear and well thought out.
- The user journeys (beneficiary, merchant, NGO) made sense.
- A working SMS-based interaction model was already defined.

However, from a technical and pilot-readiness perspective, several areas required deeper work:

- Critical security guarantees were still partially enforced off-chain.
- Smart contract logic needed clearer separation of responsibilities and roles.
- Operational risks (retries, concurrency, key handling) had not yet been stress-tested.
- The team was building toward a demo, but not yet toward something deployable in a humanitarian pilot.

The mentoring goal was therefore not to change the idea, but to harden it.

2. Detailed Mentoring Sessions

Session 1 – Full Architecture Walkthrough & Trust Boundaries

Session Goal:
Establish a shared understanding of the full system and identify where trust is currently placed.

What was reviewed

- Aiken smart contracts (aid locking and consumption)
- LTW backend service
- Merchant web platform
- SMS gateway and OTP flow

- Role definitions: NGO/Admin, Beneficiary, Merchant, Steward

Key discussion points

- Which parts of the system *must* be trusted
- Which parts *should not* be trusted
- What assumptions are safe in a humanitarian context

Observations

- Some rules (who can spend funds, when they can be spent) were implicitly enforced by backend logic.
- Time constraints (claim windows) were conceptually defined but not consistently enforced on-chain.
- SMS was being treated as both a UX and a trust mechanism.

Mentor feedback

> If the backend or SMS layer is compromised, the system should still behave safely. That's the bar for aid systems.

Decisions made

- Treat SMS strictly as a **user interface**, never as a security boundary.
- Move all irreversible rules into the Aiken validator.
- Explicitly document trust assumptions per component.

Session 2 – Smart Contract Roles & Redeemer Design

Session Goal:
Make role separation and permissions explicit and enforceable.

What was reviewed

- Redeemer structure in Aiken
- Signature checks
- Role-specific actions (Assign, Consume, Recover)

Key discussion points

- Who is allowed to do what, and under which conditions

- How to prevent “creative misuse” of redeemers
- What happens if someone tries to act outside their role

Observations

- Initial contract design allowed some ambiguity around action intent.
- Recovery logic could be accidentally triggered under the wrong conditions.
- Error paths were not fully deterministic.

Mentor feedback

> A redeemer should read like a contract clause. If it’s ambiguous, someone will eventually abuse it.

Decisions made

- Separate redeemers cleanly by action type.
- Require explicit signatures for each role:
    - Beneficiary for Consume
    - Steward for Recover
- Make invalid role/action combinations fail fast.

Session 3 – Time Locks, Expiry, and Fund Recovery

Session Goal:
Ensure time-based logic behaves safely under all conditions.

What was reviewed

- claim_start and claim_end handling
- Slot-time assumptions
- Recovery behavior after expiry

Key discussion points

- What happens if a beneficiary tries too early
- What happens if they try too late
- How NGOs recover unused funds without manual intervention

Observations

- Edge cases around timing were not fully covered by tests.
- Recovery logic needed clearer separation from consumption logic.
- There was a risk of “silent failure” without clear rejection reasons.

Mentor feedback

> Time is one of the hardest things to get right on-chain. You don’t want surprises here.

Decisions made

- Enforce all time constraints directly in the validator.
- Explicitly reject:
    - Consume before claim_start
    - Consume after claim_end
    - Recover before claim_end
- Add tests for every time-based failure path.

Session 4 – Backend Flows, OTPs, and Idempotency

Session Goal:
Stress-test the backend under real operational conditions.

What was reviewed

- Order creation and completion flow
- OTP lifecycle
- Retry behavior
- API idempotency

Key discussion points

- What happens if the same request is sent twice
- What happens if a transaction fails mid-flow
- How OTP verification relates to on-chain settlement

Observations

- Backend retries could accidentally trigger duplicate on-chain actions.

- OTP verification and settlement were too tightly coupled.
- Order state needed clearer transitions.

Mentor feedback

Assume your backend will retry, crash, or restart at the worst possible moment.

Decisions made

- Separate “order intent” from “on-chain settlement”.
- Introduce idempotency checks for settlement-triggering endpoints.
- Ensure OTP validation alone does not trigger irreversible actions.

Session 5 – UTxO Management & Concurrency

Session Goal:
Prepare the system for multiple beneficiaries and merchants operating in parallel.

What was reviewed

- Steward wallet structure
- UTxO usage patterns
- Transaction contention risks

Key discussion points

- Assigning aid to many beneficiaries at once
- Using one large UTxO vs many smaller ones
- Handling parallel transactions safely

Observations

- Using a single UTxO would quickly become a bottleneck.
- Transaction failures could cascade under load.

- Some assumptions only held for very small pilots.

Mentor feedback

Concurrency issues don’t show up in demos. They show up on day one of a pilot.

Decisions made

- Split steward funds across multiple UTxOs.
- Avoid reusing the same UTxO for concurrent assignments.
- Document recommended wallet management practices for pilots.

Session 6 – Testnet Review & Pilot Readiness Assessment

Session Goal:
Evaluate whether the system is ready for a controlled humanitarian pilot.

What was reviewed

- Preview testnet transaction results
- CPU and memory usage
- Negative-path behavior
- Operational risks still outstanding

Observations

- Assign, Consume, and Recover flows executed successfully on testnet.
- All critical rejection cases behaved as expected.
- Remaining risks were operational rather than architectural.

Mentor feedback

> The system now fails in the right ways. That's what makes it trustworthy.

Conclusions

- Core security and lifecycle rules are correctly enforced on-chain.
- The backend behaves predictably under expected failure modes.
- The system is suitable for a **limited-scope humanitarian pilot**, provided:
    - Monitoring and logging are in place
    - Key management procedures are strictly followed
    - Pilot scale is increased gradually

3. Overall Mentor Assessment

xCapit showed strong responsiveness to feedback and a clear willingness to refactor when risks were identified. Over the course of Cohort 2, the project moved from a promising concept to a technically sound prototype with real-world constraints properly accounted for.

Strengths

- Clear understanding of beneficiary realities
- Strong collaboration and openness to critique
- Good separation of concerns between on-chain and off-chain logic

Remaining Risks

- Operational complexity during live pilots
- Key custody and incident response processes
- Scaling beyond initial pilot size

Final Assessment

xCapit is technically ready for a UNDP-led pilot deployment, assuming operational safeguards are applied and pilot scope is controlled.